Quality, Architecture & Agile

IT Product Development

Innopolis University, 2026

Agenda

- Product quality
- Quality requirements
- Architecture
- Agile
- Scrum
- Product Backlog

Product Quality

Quiz

What are examples of fast software?

What are examples of slow software?

How to measure the speed of software?

Who decides which speed is enough?

Stakeholders and Quality

[Sbok5]:

Stakeholders are all individuals and groups affected by the Scrum project.

Your stakeholders are course instructors, your customer, team, users of your product.

You should identify, manage, and meet their requirements and expectations about the quality of your product (e.g. security, speed) and project (e.g. time, budget, processes, etc.) to achieve success.

Product Quality

[Iso25010]:

The **quality of a system** is the degree to which the system satisfies the stated and implied needs of its various stakeholders, and thus provides value.

Stakeholders' needs (functionality, performance, security, maintainability, etc.) are precisely what is represented in the **quality model**, which categorizes the product quality into **characteristics** and **sub-characteristics** (a.k.a "quality attributes").

The ISO 25010 Quality Model

[Iso25010]:

Quality → Characteristics → Sub-characteristics

SOFTWARE PRODUCT QUALITY								
FUNCTIONAL SUITABILITY	PERFORMANCE EFFICIENCY	COMPATIBILITY	INTERACTION CAPABILITY	RELIABILITY	SECURITY	MAINTAINABILITY	FLEXIBILITY	SAFETY
FUNCTIONAL COMPLETENESS FUNCTIONAL CORRECTNESS FUNCTIONAL APPROPRIATENESS iso25000.com	TIME BEHAVIOUR RESOURCE UTILIZATION CAPACITY	CO-EXISTENCE INTEROPERABILITY	APPROPRIATENESS RECOGNIZABILITY LEARNABILITY OPERABILITY USER ERROR PROTECTION USER ENGAGEMENT INCLUSIVITY USER ASSISTANCE SELF-DESCRIPTIVENESS	FAULTLESSNESS AVAILABILITY FAULT TOLERANCE RECOVERABILITY	CONFIDENTIALITY INTEGRITY NON-REPUDIATION ACCOUNTABILITY AUTHENTICITY RESISTANCE	MODULARITY REUSABILITY ANALYSABILITY MODIFIABILITY TESTABILITY	ADAPTABILITY SCALABILITY INSTALLABILITY REPLACEABILITY	OPERATIONAL CONSTRAINT RISK IDENTIFICATION FAIL SAFE HAZARD WARNING SAFE INTEGRATION

The Q42 Quality Model

[Q42]:

The arc42 quality model Q42 is a simple, pragmatic yet effective approach to **product and system quality**. It starts with **stakeholders' expectations** and **requirements**. From these, it derives 8 important **system properties**, which are sufficient to cover most, if not all, required, desirable or expected of the more than 100 *traditional* qualities.



Quality Requirements

How to Measure Your Product Quality? Idea 1

You ask stakeholders to assess the quality of a product version (“I know it when I see it.”).

- **Pros:**
  - Stakeholders know their needs and thus are the perfect assessors.
- **Cons:**
  - It’s probably too late. You should’ve agreed on the quality in advance to have shared expectations and to build adequate quality into your product and processes [Pmbok7].

How to Measure Your Product Quality? Idea 2

You use quality attributes from an existing model (“I don’t want to think.”).

- **Pros:**
  - Don’t need to design your own quality attributes.
- **Cons:**
  - Existing attributes are too abstract and can’t be directly measured. Example [Iso25010]:

    Capacity - Degree to which the maximum limits of a product or system parameter meet requirements.

How to Measure Your Product Quality? Idea 3

You agree with stakeholders on testable (pass/fail) quality requirements.

- **Pros:**
  - These requirements can be tested by all stakeholders (your team, your customer, etc.)
  - Their price can be estimated by tracing these requirements to related architectural decisions.
  - They can be put into a contract (“service-level agreements”).
- **Cons:**
  - They may not cover the perceived quality [Garvin].

Quality Attribute Scenarios (Quality Requirements, QR)

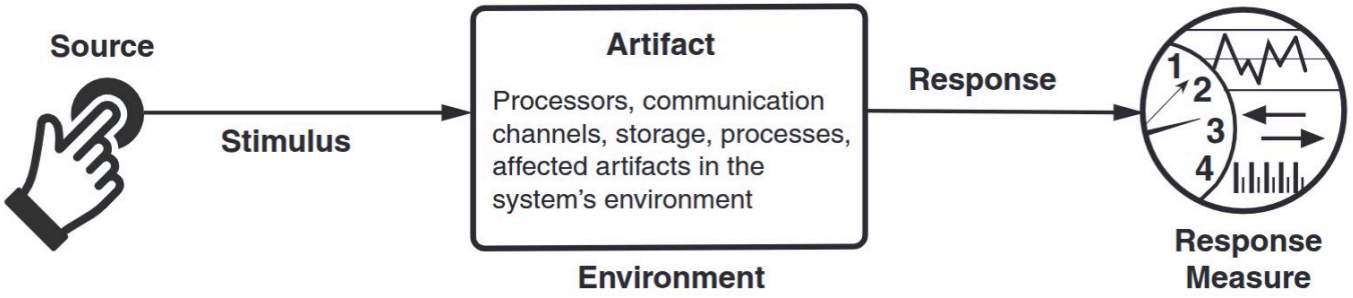
Quality attribute scenarios (**QAS**) are a way to write unambiguous testable (pass/fail) **quality requirements** (**QR**) [Bck].

To prove that the requirement is testable, you define a **quality requirement test** (**QRT**) which specifies how to test the requirement.

A scenario has 6 parts [QasArch]:

Part	Short description
Source of stimulus	An entity that generates the stimulus (human, external system, sensor, etc.)
Stimulus	A condition (event) to be considered when it arrives at a system
Environment	A system's condition when a stimulus occurs
Response	The activity undertaken at the arrival of the stimulus
Artifact	Some artifact that is stimulated; may be the whole system or part of it
Response measure	The response to the stimulus should be measurable someway so that the quality requirement can be tested

A General Quality Attribute Scenario for Availability



Source	Stimulus	Environment	Response	Response Measure
Internal/external: people, hardware, software, physical infrastructure, physical environment	Fault: omission, crash, incorrect timing, incorrect response	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation	Prevent the fault from becoming a failure Detect the fault Recover from the fault	Time or time interval when the system must be available Availability percentage Time to detect the fault

FIGURE 3.2 A general scenario for availability

Source: [Bck]

Example: A QR for Moodle's Availability

Source of Stimulus	External: A large number of students (e.g., 500+) simultaneously using the system.
Stimulus	Fault (Crash): An unexpected number of concurrent database write operations, triggered by all students submitting a high-stakes quiz in the final minute before the deadline, causes the primary database server to crash.
Artifact	The Moodle application, specifically the database server and the services that depend on it (e.g., quiz submission, gradebook, session management).
Environment	Normal (Peak) Operation: The system is operating under a heavy but expected load on a weekday evening, which is a known peak usage time for students.
Response	The system’s high-availability mechanism detects the primary database failure. It automatically initiates a failover to a redundant, standby database server. A brief maintenance page is displayed to users currently in the process of submitting, and the system then seamlessly restores their sessions once the failover is complete.
Response Measure	Full service is restored within 3 minutes. No more than 0.5% of student quiz submissions are lost and require resubmission. The system’s availability must not drop below 99.9% during the academic term.

Example: A QR test

- **Step 1: Simulate the Stimulus**
  - In a staging environment, we use a load testing tool (JMeter) to simulate **500+ students** submitting a quiz at the exact same second.
- **Step 2: Inject the Fault**
  - At the precise moment of peak submission load, we intentionally **crash the primary database server** to trigger the automated failover system.
- **Step 3: Measure the Response**
  - The test is considered a **PASS** only if the system automatically recovers and meets these strict criteria:
    - **Recovery Time:** Full functionality is restored in **under 3 minutes**.
    - **Data Integrity:** Fewer than **0.5%** of student submissions are lost.
    - **Availability:** The overall uptime of **99.9%** is maintained.

How to obtain QAS?

Quality requirements belong to **architectural drivers** that also include primary functional requirements, technical and business constraints, business goals. More about them later.

To obtain QAS, you can conduct a **Quality Attribute Workshop** (**QAW**, more info here).

During QAW, facilitators (team members, AI) identify and clarify architectural drivers with stakeholders and the architect and then help stakeholders generate, prioritize, verify (consistency, feasibility, etc.) and refine QA scenarios based on these drivers.

Key Takeaways

- Elicit quality requirements at the project start to better manage customer expectations.
- Use quality attribute scenarios to express testable quality requirements.
- Derive specific quality requirements from generic quality attributes in existing quality models.
- Use AI to produce requirements and specify corresponding tests.
- More info: Quality

Architecture

Quiz

Which software systems have an architecture?

What is the architecture of a software system?

What is the Architecture of a Software System?

[Bck]:

The **software architecture** of a system is the set of **structures** needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.

A structure is **architectural** if it supports reasoning about the system and the system's quality attributes. The reasoning should be about attributes of the system that are important to some stakeholder(s).

The attributes include **functionality** achieved by the system (see Functional Suitability in [Iso25010]).

Why is the Architecture Important?

[Bck]:

- The architecture connects (often abstract) business goals and requirements with the (concrete) tasks and implementation.
- Designing an architecture helps avoid costly changes later.
- An architecture will inhibit or enable a system’s driving quality attributes.
- A documented architecture enhances communication among stakeholders.
- The decisions made in an architecture allow you to reason about and manage change as the system evolves.
- An architecture defines a set of constraints on subsequent implementation.

What can Affect the Architecture?

[Bck]:

The architecture is developed based on **Architecturally Significant Requirements** (ASRs) that can affect quality attributes of the system.

ASRs are:

- Primary functional requirements;
- Quality requirements;
- Business goals;
- Constraints (business, technical)

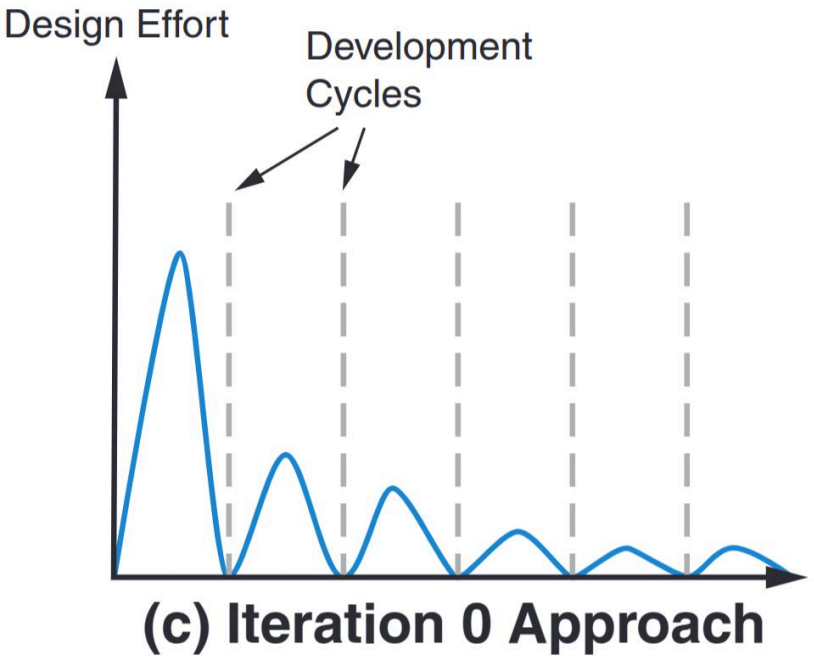
When to Design the Architecture?

[Bck]:

Design mostly at the very beginning of the project.

Continue designing and detailing the architecture throughout the project.

Change the architecture in response to changing requirements.



How to Design an Architecture?

See:

- Designing an Architecture [Bck, Ch. 20]
- An LLM-assisted approach to designing software architectures using ADD [LlmAdd]

How to Document the Architecture?

See [Bck, Chapter 22], Example 1, Example 2

Diagrams:

- Static views (context diagram, component diagram)
- Dynamic views (sequence diagram)

The diagrams must include a key (legend)!

Text:

- **Architectural decision records** (**ADR**) - architectural decisions with justification; records have a uniform format
- Diagram descriptions

Key Takeaways

- The architecture should mostly be developed early in the project.
- The architecture depends on architecturally significant requirements (ASRs).
- The ASRs should be gathered early in the project.
- The ASRs include quality requirements.
- Hence, quality requirements should be gathered early in the project.
- More info: Architecture

Agile

Quiz

When did Agile appear?

Is Agile an excuse to not deliver results on time (true/false)?

Agile Manifesto

[AgileManifesto]:

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Agile and Architecture

See Principles behind the Agile Manifesto, Agile and architecture [Bck]:

Agile Principle	Agree?	Architecture-centric View
The best architectures, requirements, and designs emerge from self-organizing teams.	No	No, they don't. The best architectures are consciously designed by skilled, talented, trained, and experienced architects, as we describe in Chapter 20
The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.	No	This is nonsense for nontrivial systems. Humans invented writing because our brains can't remember everything we need to remember.
Business people and developers must work together daily throughout the project.	Mostly	Business goals lead to quality attribute requirements, which the architecture's primary duty is to fulfill, as we discussed in Chapter 19.
Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.	Mostly	While we agree in principle, many developers are inexperienced. So make sure to include a skilled, experienced, and motivated architect to help guide these individuals.

Working software is the primary measure of progress.	Mostly	Yes, as long as “primary” is not taken to mean “only,” and as long as this principle is not used as an excuse to eliminate all work except coding.
Simplicity — the art of maximizing the amount of work not done — is essential.	Mostly	Yes, of course, as long as it is understood that the work we are not doing can actually be jettisoned safely without detriment to the system being delivered.
Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter time scale.	Yes!	Absolutely, as long as this principle is not seen as precluding a thoughtful architecture. DevOps has a large role to play here, and we have seen, in Chapter 5, how architectures can support DevOps.
Welcome changing requirements, even late in development. Agile processes harness change for the customer’s competitive advantage.	Yes!	Absolutely. This principle is served by architectures that provide high degrees of modifiability (Chapter 8) and deployability (Chapter 5).

Does architecture-centric view agree with other principles? **Yes!**

Why still use Agile for the ITPD Course?

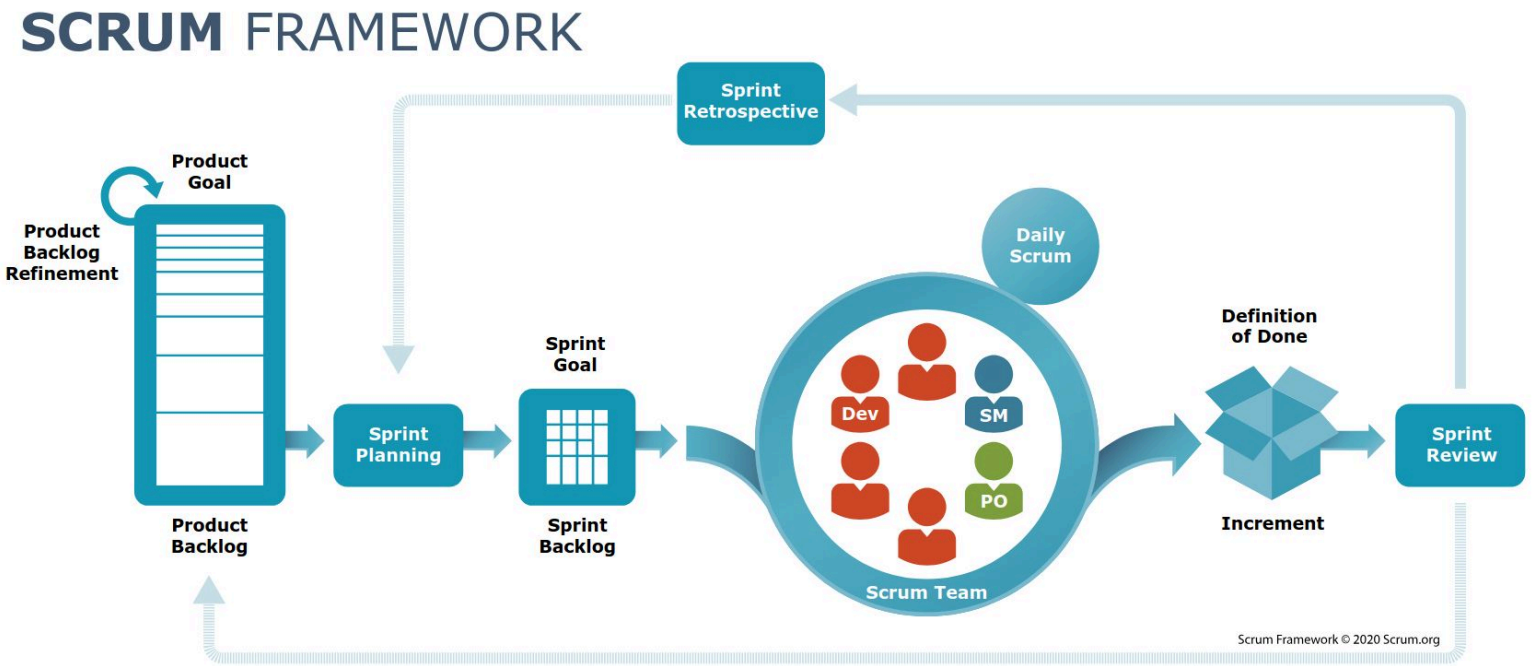
- You work in a team so you need to care about interactions with your teammates.
- The ultimate goal is to produce working valuable software although we also require you manage the configuration to be able to comprehensively control the project.
- We want you to learn to elicit and analyze requirements so you have to meet with the customer weekly. We don’t provide a contract to negotiate.
- The development of a new product can be unpredictable so you have to respond to changes although you should also keep in mind the general plan to know how it changes when changes arrive.

Scrum

Quiz

- What is Scrum?
- How is it related to Agile?

Scrum Framework (visualized)



Definitions

[ScrumGuide]

- **Scrum** - a lightweight process framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.
- **Product Backlog** - emergent, ordered list of what is needed to improve the product. It is the single source of work undertaken by the Scrum Team. Consists of **Product Backlog items (PBI)** - user stories, tasks, etc.
- **Sprint** - a container for Scrum events (**Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective**). *1 week long for this course.*
- **Sprint Backlog** - the **Sprint Goal**, the PBIs selected for the Sprint (**Sprint Backlog items** - **SBI**), plus the plan for delivering them.
- **Sprint Goal** - communicates why the Sprint is valuable to stakeholders. Defined at the Sprint Planning.

Scrum events

- **Sprint Planning (start of the Sprint)** - The team plans the work to be performed for the Sprint. They define the Sprint Goal and select the most important PBIs. Developers decompose the PBIs into smaller work items.
- **Daily Scrum** - A daily 15-minute meeting where the team inspects the progress towards the Sprint Goal and produces an actionable plan for the next day of work.
- **Sprint Review (near the end of the Sprint)** - The team inspects the outcome of the Sprint with the customer and determines future adaptations.
- **Sprint Retrospective (after the Sprint Review)** - The team plans ways to increase quality and effectiveness.

Source: The Scrum Guide

Scrum in ITPD

In ITPD, we use elements of Scrum that align well with what's imporant in the course - regular communication inside the team and with the customer, iterative progress towards MVP, bounded amount of work each Sprint.

Please don't tell during the Industrial Project that you were trained to use Scrum in ITPD!

As explained in the [ScrumGuide] end note:

> "The Scrum framework, as outlined herein, is immutable. While implementing only parts of Scrum is possible, the result is not Scrum. Scrum exists only in its entirety and functions well as a container for other techniques, methodologies, and practices."

Key Takeaways

Key Takeaways

- You must think about quality early if you want to have control over your software system via its architecture.
- Effective agile practices should be supported by the well-thought-out architecture and configuration management.
- Scrum is a well-documented framework in spirit of Agile that guides you how to deliver every Sprint.

References

1. [Sbok5]: SBOK® Guide, 5th ed., https://www.scrumstudy.com/sbokguide/download-free-buy-sbok#new-sbok-download
2. [Iso25010]: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010
3. [Pmbok6]: PMBOK® Guide, 6th ed., https://www.pmi.org/standards/pmbok
4. [Pmbok7]: PMBOK® Guide, 7th ed., https://www.pmi.org/standards/pmbok
5. [Q42]: https://quality.arc42.org/
6. [Garvin]: https://en.wikipedia.org/wiki/Eight_dimensions_of_quality
7. [QasArch]: Quality attributes of software architectures https://www.cs.unibo.it/cianca/wwwpages/as/5.pdf#page=26
8. [Bck]: L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed.
9. [LlmAdd]: An LLM-assisted approach to designing software architectures using ADD, https://arxiv.org/abs/2506.22688
10. [AgileManifesto]: Manifesto for Agile Software Development https://agilemanifesto.org/
11. [ScrumGuide]: Scrum Guide™ https://scrumguides.org/scrum-guide.html